PROJECT REPORT

ON

Quizzkr – Online Quiz Platform

(Advanced anti-cheat quiz platform - using DBMS)

Submitted by

Nikunj Miglani (Reg No.RA2411030030043)

Under the guidance of

Dr. Anirudha Gaikwad

(Assistant Professor, Department of Computer Science & Engineering)

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

of

FACULTY OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)
DELHI-NCR CAMPUS, GHAZIABAD (U.P)

SRM INSTITUTE OF SCIENCE & TECHNOLOGY,

DELHI-NCR CAMPUS

April 2026

(Under Section 3 of UGC Act, 1056)

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "**Project Report on Quizzkr – Online Quiz Platform**" submitted by **Nikunj Miglani (Reg No.RA2411030030043** of **B.Tech — Computer Science and Engineering with specialization in Cyber Security**, in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering**, is a record of the original work carried out by them under my supervision and guidance.

This project has been carried out during the academic year **2025–2026** at **SRM Institute of Science and Technology, Delhi-NCR Campus**, and represents the sincere efforts of the students. To the best of my knowledge and belief, this project has not been submitted, in part or full, for the award of any other degree or diploma at this institute or any other educational institution.

The project report embodies the results of investigations conducted by the students and demonstrates their understanding of the subject matter, their ability to carry out systematic research, and their capacity for critical analysis and independent thinking.

I am confident that the knowledge and experience gained by the students through this project will prove valuable for their academic and professional growth in the field of computer science and software development.

I further certify that all necessary ethical guidelines and academic standards have been followed in the preparation of this project report.

SIGNATURE

(Dr. Anirudha Gaikwad)
GUIDE
Assistant Professor
Dept. of Computer Science & Engineering
SRM Institute of Science & Technology

SIGNATURE

(Dr. Neelam Singh)
HEAD OF DEPARTMENT (CSE)
Dept. of Computer Science & Engineering
SRM Institute of Science & Technology

ACKNOWLEDGEMENT

I take this opportunity to express my profound gratitude and deep regards to all those who have extended their invaluable assistance and support in the successful completion of my project titled **"Project Report on Quizzkr – Online Quiz Platform"**.

First and foremost, I am deeply indebted to my esteemed guide, **Dr. Anirudha Gaikwad**, Assistant Professor, Department of Computer Science and Engineering, SRM Institute of Science and Technology, Delhi-NCR Campus, for his constant supervision, insightful guidance, and constructive feedback throughout the course of this project. His patience, motivation, and immense knowledge have been pivotal in shaping this work into its final form.

I am equally grateful to **Dr. Neelam Singh**, Head of Department, CSE for her encouragement, academic support, and for providing me with the opportunity and resources to undertake this project successfully.

I extend my sincere thanks to the faculty members and technical staff of the **Department of Computer Science and Engineering**, SRM Institute of Science and Technology, for their cooperation and assistance, which contributed immensely to the smooth execution of my project work.

I would also like to express my heartfelt appreciation to my **parents** and **friends**, whose unwavering support and encouragement kept me motivated throughout this journey. Their belief in my abilities gave me the strength to face challenges and accomplish my goals.

Lastly, I extend my gratitude to all the contributors of online resources and learning platforms such as **GeeksforGeeks**, **W3Schools**, **ChatGPT** and other references that enhanced my understanding and helped me overcome technical difficulties during project development.

This project has been a rewarding learning experience, enabling me to enhance my technical skills, deepen my knowledge of Database Management System and algorithmic thinking, and appreciate the importance of systematic design. I am truly thankful to all who, directly or indirectly, helped make this project a success.

I am truly thankful to all who, directly or indirectly, helped me make this project a success.

TABLE OF CONTENT

Chapters	Pages No.
1. Introduction	5
2. Problem Statement	6-7
3. System Architecture and Design	8-9
4. Database Schema and Design	10-13
5. Environment Configuration	14
6. Technology Stack	15
7. API Endpoints	16
8. Source Code (Key Modules)	17–20
9. User Flows and Working of the System	21-22
10. Result and Output	23-29
11. Advantages and Limitations	30
12. Future Enhancements	31
13. Conclusion	32
14. References	33

Chapter 1 — Introduction

With the increasing shift toward digital learning and remote assessments, there is a growing need for systems that can conduct quizzes in a structured, secure, and scalable manner. Many existing platforms either lack proper backend control over evaluation or fail to implement mechanisms that ensure fairness during assessments. This project addresses these gaps by developing a full-stack online quiz platform with controlled evaluation flow and integrated monitoring features.

The system is built using **Next.js** as a unified full-stack framework, enabling seamless integration of frontend and backend through API route handlers. **A PostgreSQL** database is used for persistent storage, with **Prisma ORM** handling schema management and database interactions in a type-safe manner. Authentication is implemented using **NextAuth**, supplemented with custom endpoints for user registration and secure password reset via email-based token verification.

The platform follows a role-based architecture, supporting three distinct roles: User, Creator, and Admin. Each role interacts with the system through dedicated APIs and dashboards. Creators can design and publish quizzes, users can attempt them under controlled conditions, and administrators manage user roles and system access.

A key design decision in this system is the server-side control of the entire quiz lifecycle. Quiz attempts are initiated, tracked, and evaluated on the backend, ensuring that scoring logic cannot be manipulated from the client side. Each attempt records timestamps, scores, and behavioral indicators, forming a reliable audit trail for evaluation.

To improve assessment integrity, the platform incorporates basic anti-cheating detection mechanisms, including tab-switch monitoring and abnormal submission time analysis. Attempts flagged under these conditions are marked as suspicious for further review. In addition, the system provides dashboards for performance tracking and historical analysis of quiz attempts.
Overall, this project demonstrates the practical implementation of database design, API architecture, authentication systems, and controlled evaluation workflows in building a reliable online assessment platform.

Chapter 2 — Problem Statement

2.1 Problems of Existing Quiz Systems

Most basic online quiz platforms suffer from several architectural and security limitations that reduce their reliability in real-world assessment scenarios. These issues are outlined below:

- Lack of Role-Based Access Control:

  Many systems do not differentiate between types of users. There is no structured separation between quiz creators, participants, and administrators, which leads to poor access control and limited scalability in multi-user environments.

- Client-Side Evaluation Vulnerability:

  In several implementations, scoring logic is partially or fully handled on the frontend. This makes the system vulnerable to manipulation using browser developer tools, allowing users to alter scores or bypass validation.

- Absence of Attempt Tracking Mechanisms:

  Traditional platforms often fail to maintain a proper lifecycle for quiz attempts. There is no structured tracking of when an attempt starts, ends, or how it progresses, making it difficult to audit or analyze user performance.

- No Detection of Suspicious Behaviour:

  Most systems lack mechanisms to detect irregular user behavior during quizzes, such as tab switching or unusually fast submissions. This creates opportunities for unfair practices during assessments.

- Weak Authentication and Account Recovery Systems:

  Basic implementations often lack secure password reset workflows and rely on minimal authentication logic, which can compromise user account security.

- Poorly Structured Database Design:

  Some quiz systems use unstructured or loosely designed schemas without proper relationships or indexing, leading to inefficient queries, data redundancy, and scalability issues.

2.2 Proposed Solution

To address the above limitations, this project implements a structured, backend-controlled online quiz platform with emphasis on security, data integrity, and role-based interaction.

- Role-Based System Architecture:

  The platform introduces three distinct roles—User, Creator, and Admin—with clearly defined permissions enforced at both API and database levels.

- Server-Side Evaluation Pipeline:

  All scoring and validation logic is handled on the backend. The client only submits selected answers, ensuring that results cannot be manipulated from the frontend.

- Structured Attempt Lifecycle Management:

  Each quiz attempt is tracked with timestamps, score, and completion status, enabling accurate monitoring and historical analysis.

- Basic Anti-Cheating Detection:

  The system incorporates behavioral checks such as tab-switch detection and submission time analysis. Attempts identified as abnormal are flagged as suspicious for further review.

- Secure Authentication and Recovery:

  Authentication is handled using NextAuth, along with a token-based password reset mechanism implemented using email verification.

- Relational Database Design with Prisma:

  The database is designed using a normalized relational schema with clearly defined relationships between users, quizzes, questions, and attempts. Indexing is applied on key fields to optimize performance.

Chapter 3 — System Architecture and Design

3.1 Overall Architecture

The system follows a monolithic full-stack architecture built using the Next.js App Router. Both frontend and backend logic are implemented within a single codebase, where React components handle the user interface and API route handlers manage server-side operations. This approach simplifies deployment while maintaining a clear separation between presentation and business logic.

The architecture is divided into three primary layers:

Presentation Layer:

The frontend is built using React components within Next.js. It includes role-based dashboards, quiz interfaces, authentication pages, and result views. Tailwind CSS is used for styling to ensure a responsive and consistent user interface.

Application Layer:

The backend consists of Next.js API route handlers that process all business logic. These APIs handle authentication, quiz creation, attempt management, scoring, and user operations. All critical operations such as evaluation and validation are executed on the server to ensure security and integrity.

Data Layer:

The system uses PostgreSQL as the relational database, accessed through Prisma ORM. The database stores structured data for users, quizzes, questions, attempts, and password reset tokens. Relationships between entities are enforced using foreign keys, and indexing is applied to optimize query performance.

3.2 Application Structure

The project follows a modular structure aligned with Next.js App Router conventions. The key components of the system are:

API Layer (/api):
Contains all backend endpoints, including:

Authentication (signup, login, password reset)
Quiz management (create, fetch, leaderboard)
Attempt handling (start, submit, fetch)
User dashboards and history
Role management (admin operations)

Frontend(/app):
Includes all user-facing pages such as login, registration, dashboards, quiz attempt interfaces, and result views. Routing is handled dynamically using Next.js file-based routing.

Database/prisma):
Contains the Prisma schema which defines all database models and relationships. It serves as the single source of truth for database structure and is used to generate type-safe queries.

Utility(/lib):
Includes reusable logic such as database connection setup, authentication helpers, and email handling for password reset functionality.

3.3 Authentication and Authorization Flow

Authentication is handled using NextAuth, which manages user sessions securely. Users can register using email and password, and credentials are validated during login.

For password recovery, a token-based reset mechanism is implemented:

A unique token is generated and sent to the user via email using **Nodemailer**.
The token has an expiration time and is validated before allowing password reset.

Authorization is enforced using a role-based access control system:

Each user is assigned a role (User, Creator, Admin).
API routes check user roles before granting access to protected operations such as quiz creation or admin actions.

3.4 Request Flow (High-Level)

The system follows a structured request-response lifecycle:

User interacts with the frontend (e.g., starts a quiz).
A request is sent to the corresponding API endpoint.

The backend validates the request and interacts with the database via Prisma.
Business logic (such as scoring or validation) is executed on the server.

A response is returned to the frontend and rendered to the user.

This architecture ensures that all critical operations remain under server control, reducing the risk of manipulation and maintaining consistency across the system.

Chapter 4 — Database Schema and Design

4.1 Database Technology

The system uses PostgreSQL as the relational database for storing all application data. PostgreSQL is chosen for its reliability, support for relational integrity, and strong indexing capabilities.

Database interaction is handled using Prisma ORM, which provides a type-safe interface for querying and managing the database. The database connection is configured using an environment variable (DATABASE_URL), ensuring secure and flexible deployment.

4.2 Schema Design Overview

The database is designed using a relational model with normalized structure, ensuring minimal redundancy and strong data consistency.

The schema consists of five core entities:

User

Quiz

Question

Attempt

PasswordResetToken

Each entity is connected through well-defined relationships using foreign keys, enabling efficient querying and maintaining referential integrity.

4.3 Table Descriptions

1. User

Stores all registered users of the platform.

Key Fields:

id (Primary Key)

name

email (Unique)

password

role (Enum: USER, CREATOR, ADMIN)

createdAt, updatedAt

Relationships:

One user can create multiple quizzes

One user can have multiple quiz attempts

2. Quiz

Stores metadata for quizzes created by users.

Key Fields:

id (Primary Key)

title

description

creatorId (Foreign Key → User)

createdAt

Relationships:

One quiz belongs to one creator

One quiz contains multiple questions

One quiz can have multiple attempts

3. Question

Stores individual quiz questions.

Key Fields:

id (Primary Key)

question (text)

options (JSON format)

answer (correct answer)

quizId (Foreign Key → Quiz)

Design Insight:
Options are stored as JSON, allowing flexible representation of answer choices without requiring a separate table.

4. Attempt

Tracks each user's attempt for a quiz.

Key Fields:

id (Primary Key)

userId (Foreign Key → User)

quizId (Foreign Key → Quiz)

score

total

suspicious (Boolean flag)

startedAt, endedAt

createdAt

Relationships:

Many attempts belong to one user

Many attempts belong to one quiz

Design Importance:
This is the central table for evaluation, storing both performance and behavioral indicators.

5. PasswordResetToken

Handles password reset functionality.

Key Fields:

id (Primary Key)

token (Unique)

email

expiresAt

createdAt

4.4 Relationships and Constraints

The schema enforces relationships using foreign keys:

User → Quiz (one-to-many)

Quiz → Question (one-to-many)

User → Attempt (one-to-many)

Quiz → Attempt (one-to-many)

Constraints:

Unique constraint on user email

Unique constraint on reset token

Cascade delete on related entities (e.g., deleting a quiz removes its questions and attempts)

4.5 Indexing Strategy

Indexes are applied on frequently queried fields to improve performance:

User.role → for role-based queries

Quiz.creatorId → for fetching creator-specific quizzes

Quiz.createdAt → for sorting and filtering

Attempt.userId and Attempt.quizId → for retrieving attempts efficiently

Attempt.score and Attempt.createdAt → for analytics and history queries

Question.quizId → for fast question retrieval

4.6 Design Considerations

The database design follows key DBMS principles:

Normalization: Data is structured to avoid redundancy

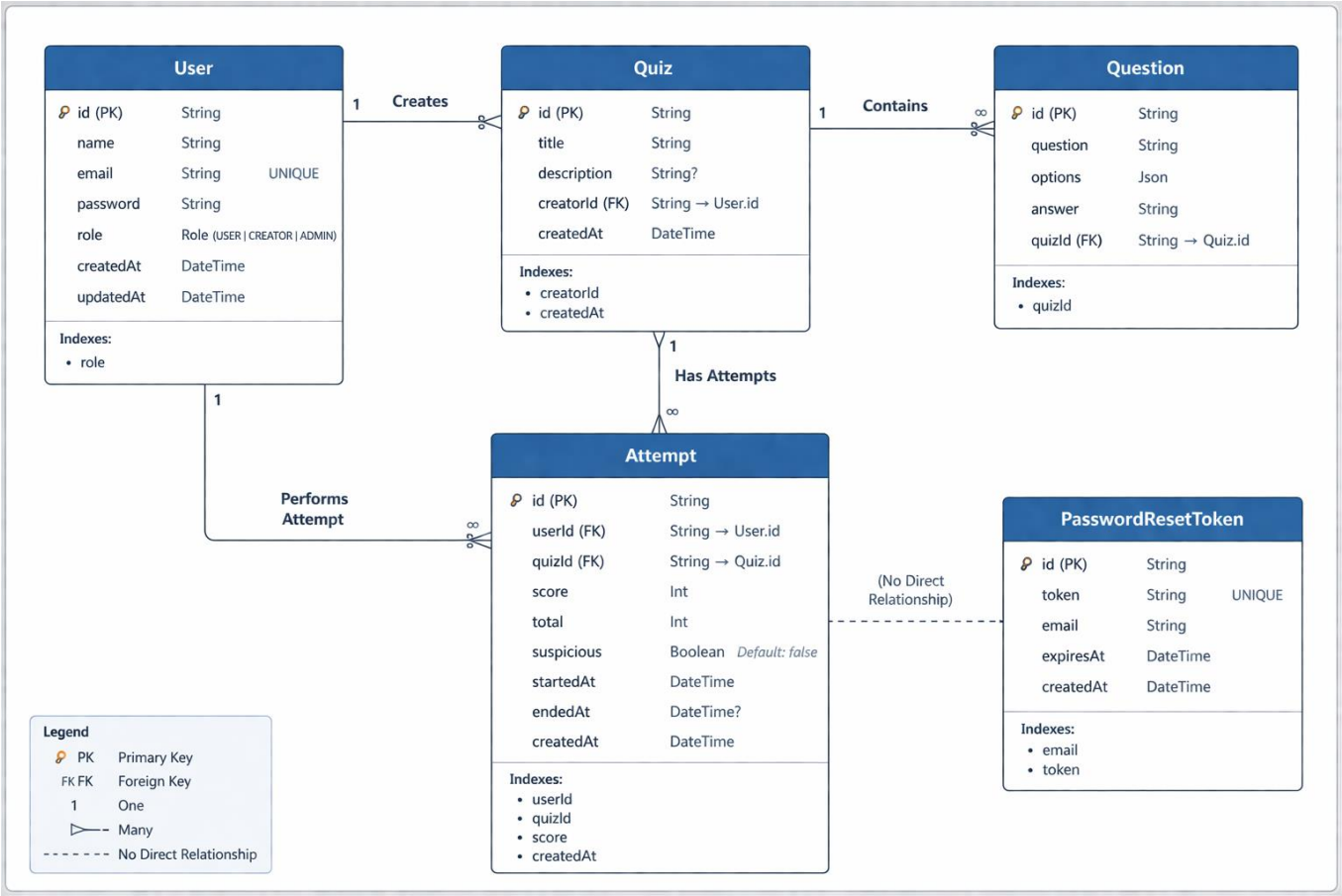
Referential Integrity: Enforced using foreign keys

Scalability: Indexed fields support efficient querying at scale

Flexibility: JSON-based options allow dynamic question formats

Security: Sensitive operations like scoring are handled outside the database in controlled backend logic

4.7 ER Diagram



Chapter 5 — Environment Configuration

The application requires a set of environment variables to be configured for both local development and production deployment. These variables are stored in a .env file and are used to securely manage sensitive credentials and configuration parameters.

5.1 Database Configuration

```
DATABASE_URL=""
```

The **DATABASE_URL** must be set to the Neon DB connection string in the format:

```
postgresql://[user]:[password]@[host]/[database]?sslmode=require
```

5.2 Authentication Configuration

```
NEXTAUTH_SECRET=""
NEXTAUTH_URL="http://www.quizzkr.vercel.app"
```

NEXTAUTH_SECRET is used by NextAuth to encrypt session data and ensure secure authentication.
NEXTAUTH_URL defines the base URL of the application, required for correct callback handling in authentication flows.

Authentication is handled using **NextAuth**, which manages user sessions securely across the application.

5.3 SMTP Email Configuration

```
SMTP_HOST=""
SMTP_PORT=""
SMTP_SECURE=""
SMTP_USER=""
SMTP_PASS=""
SMTP_FROM_EMAIL=""
```

These variables configure the email service used for sending password reset links.
SMTP_HOST and **SMTP_PORT** define the mail server.
SMTP_USER and **SMTP_PASS** are credentials used for authentication.
SMTP_FROM_EMAIL specifies the sender address.
The system uses **Nodemailer** to send transactional emails, particularly for password reset functionality.

Chapter 6 — Technology Stack

6.1 Frontend

- **Next.js (App Router):**

  The frontend is built using Next.js, which provides a full-stack React framework with file-based routing and support for both server and client components. It enables seamless integration between UI and backend APIs within a single codebase.

- **React:**

  Used for building interactive user interfaces, including quiz pages, dashboards, and authentication forms. React's component-based architecture allows modular and reusable UI development.

- **Tailwind CSS:**

  A utility-first CSS framework used for styling the application. It enables rapid UI development with consistent design and responsive layouts.

6.2 Backend

- **Next.js API Route Handlers:** Server-side REST endpoints for all business logic.
- **bcryptjs:** Password hashing with salted bcrypt (10 rounds).
- **Nodemailer:** SMTP-based transactional email for verification and password reset.

6.3 Database

- **PostgreSQL:** Relational database for all persistent data.
- **Neon DB:** Serverless PostgreSQL hosting with connection pooling and branching.
- **Prisma ORM:** Type-safe database client with migration support and schema introspection.

6.4 DevOps and Infrastructure

- **Vercel:** Deployment platform with Edge Functions and automatic CI/CD from GitHub.
- **Gmail SMTP:** Transactional email via Gmail with App Passwords.

Chapter 7 — API Endpoints

All API endpoints are implemented as Next.js App Router Route Handlers under the /src/app/api/ directory.

7.1 Authentication Endpoints

Method	Endpoint	Description
POST	/api/auth/signup	Register a new user, hashes password with bcrypt.
POST	/api/auth/forgot-password	Sends a password reset email with a hashed token link.
POST	/api/auth/reset-password	Resets password using a valid unexpired token hash.
GET	/api/auth/[...nextauth]	Handles Auth callback. Creates/links user, returns Token.

7.2 Quiz Management Endpoints

Method	Endpoint	Description
GET	/api/quizzes	List all quizzes.
POST	/api/quizzes/create	Create a new quiz with questions.
GET	/api/quizzes/[quizId]	Fetch quiz details including questions and options.
POST	/api/user/become-creator	Upgrades a user role to creator.

7.3 Attempt Endpoints

Method	Endpoint	Description
GET	/api/quizzes/[quizId]/attempt	Load the quiz attempt page. Returns questions (optionally shuffled) and existing draft.
POST	/api/attempts/start	Initializes a new quiz attempt and records start time.
POST	/api/attempts/submit	Submits answers, calculates score on server, and finalizes attempt.
GET	/api/quizzes/[quizId]/attempts/[attemptId]	Fetch detailed attempt review (Teacher/Admin).

7.4 Leaderboard and dashboard Endpoints

Method	Endpoint	Description
GET	/api/quizzes/[quizId]/leaderboard	Fetch ranked leaderboard. Visibility controlled by quiz settings.
GET	/api/dashboard	Returns user-specific dashboard data (stats, recent activity).
GET	/api/user/history	Fetches all quiz attempts for the logged-in user.

Chapter 8 — Source Code (Key Modules)

The complete source code is available in the attached repository. Below are the key modules and their most important code sections, demonstrating the DBMS interactions and business logic.

8.1 Prisma Client Singleton (src/lib/prisma.ts)

Prisma is instantiated as a singleton to prevent connection pool exhaustion in serverless environments:

```
import { PrismaClient } from "@prisma/client";

const globalForPrisma = global as unknown as {
prisma: PrismaClient;
};

export const prisma =
globalForPrisma.prisma || new PrismaClient();

if (process.env.NODE_ENV !== "production") {
globalForPrisma.prisma = prisma;
}
```

8.2 Authentication Utilities (src/lib/auth.ts)

```
import type { AuthOptions } from "next-auth"
import CredentialsProvider from "next-auth/providers/credentials"
import { prisma } from "@/lib/prisma"
import bcrypt from "bcrypt"

export const authOptions: AuthOptions = {
providers: [
CredentialsProvider({
name: "Credentials",
credentials: {
email: {},
password: {},
},
async authorize(credentials) {
if (!credentials?.email || !credentials?.password) return null

const user = await prisma.user.findUnique({
where: { email: credentials.email },
})

if (!user) return null

const isValid = await bcrypt.compare(
credentials.password,
user.password
)

if (!isValid) return null

return {
id: user.id,
email: user.email,
name: user.name,
role: user.role,
}
},
}),
],

session: {
strategy: "jwt",
},
```

```
callbacks: {
async jwt({ token }) {
if (token?.email) {
const user = await prisma.user.findUnique({
where: { email: token.email },
select: { id: true, role: true },
})

if (user) {
token.id = user.id
token.role = user.role
}
}

return token
},

async session({ session, token }) {
if (session.user) {
session.user.id = token.id as string
session.user.role = token.role as "USER" | "CREATOR" | "ADMIN"
}
return session
},
}
}
```

8.3 User Registration (src/app/api/auth/signup/route.ts)

```
import { prisma } from "@/lib/prisma"
import bcrypt from "bcrypt"

export async function POST(req: Request) {
const { name, email, password } = await req.json()

const hashed = await bcrypt.hash(password, 10)

const user = await prisma.user.create({
data: {
name,
email,
password: hashed,
},
})

return Response.json({ success: true, user })
}
```

8.4 Attempt Submission (src/app/api/attempts/submit/route.ts)

```
import { prisma } from "@/lib/prisma"

export async function POST(req: Request) {
try {
const body = await req.json()
const { attemptId, answers, isCheating } = body

if (!attemptId || !answers || typeof answers !== "object") {
return Response.json(
{ success: false, message: "Invalid data" },
{ status: 400 }
)
}

const attempt = await prisma.attempt.findUnique({
where: { id: attemptId },
include: {
quiz: {
include: { questions: true },
```

```
},
},
})

if (!attempt) {
return Response.json(
{ success: false, message: "Attempt not found" },
{ status: 404 }
)
}

if (attempt.endedAt) {
return Response.json(
{ success: false, message: "Already submitted" },
{ status: 400 }
)
}



const now = new Date()
const timeTaken =
(now.getTime() - attempt.startedAt.getTime()) / 1000

if (timeTaken > 60) {
return Response.json(
{ success: false, message: "Time exceeded" },
{ status: 400 }
)
}

let score = 0

for (const q of attempt.quiz.questions) {
if (answers[q.id] === q.answer) {
score++
}
}

const suspicious =
isCheating === true ||
(timeTaken < 10 && score === attempt.quiz.questions.length)

const updated = await prisma.attempt.update({
where: { id: attemptId },
data: {
score,
total: attempt.quiz.questions.length,
endedAt: new Date(),
suspicious,
},
})

return Response.json({
success: true,
score,
total: attempt.quiz.questions.length,
suspicious,
})
} catch (error) {
console.error(error)
return Response.json(
{ success: false, message: "Server error" },
{ status: 500 }
)
}
}
```

8.7 Prisma Schema (prisma/schema.prisma) — Key Tables

```
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator erd {
  provider = "prisma-erd-generator"
  output   = "./ERD.svg"
}
enum Role {
  USER
  CREATOR
  ADMIN
}

model User {
  id        String   @id @default(cuid())
  name      String
  email     String   @unique
  password  String
  role      Role     @default(USER)

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt





  attempts  Attempt[]
  quizzes   Quiz[]   @relation("CreatorQuizzes")

  @@index([role])
}

model Quiz {
  id          String     @id @default(cuid())
  title       String
  description String?

  creatorId   String
  creator     User       @relation("CreatorQuizzes", fields: [creatorId],
references: [id])

  questions   Question[]
  attempts    Attempt[]

  createdAt   DateTime   @default(now())

  @@index([creatorId])
  @@index([createdAt])
}

model Question {
  id        String   @id @default(cuid())
  question  String
  options   Json
  answer    String

  quizId    String
  quiz      Quiz     @relation(fields: [quizId], references: [id], onDelete:
Cascade)

  @@index([quizId])
}
```

```
model Attempt {
  id          String   @id @default(cuid())

  userId      String
  quizId      String

  score       Int
  total       Int

  suspicious  Boolean  @default(false)

  startedAt   DateTime @default(now())
  endedAt     DateTime?
   createdAt   DateTime @default(now())


  user        User     @relation(fields: [userId], references: [id], onDelete:
Cascade)
  quiz        Quiz     @relation(fields: [quizId], references: [id], onDelete:
Cascade)

  @@index([userId])
  @@index([quizId])
  @@index([score])
  @@index([createdAt])
}

model PasswordResetToken {
  id        String   @id @default(cuid())
  token     String   @unique
  email     String
  expiresAt DateTime
  createdAt DateTime @default(now())

  @@index([email])
  @@index([token])
}
```

Chapter 9 — User Flows and Working of the System

9.1 Creator Flow

The following steps describe how a quiz is created and managed by a Creator:

Step 1 — Registration and Login:
The user registers on the platform and logs in using credentials. Based on role assignment, the user operates as a Creator. Authentication and session handling are managed through NextAuth.

Step 2 — Create Quiz:
From the dashboard, the creator navigates to the quiz creation interface and enters quiz details such as title and description. Questions are added along with multiple options and the correct answer.

Step 3 — Store Quiz Data:
On submission, quiz data is sent to the backend via API routes. The server validates the input and stores the quiz and its associated questions in the database using Prisma.

Step 4 — Manage Quizzes:
Creators can view their created quizzes and access performance metrics such as number of attempts and average scores through the dashboard.

Step 5 — Review Attempts:
Creators can view attempts made by users, including scores and attempt details, to analyze performance.

9.2 User (Student) Flow

The following steps describe how a user interacts with the quiz system:

Step 1 — Registration and Login:
The user creates an account and logs into the system using valid credentials.

Step 2 — Browse Available Quizzes:
The user views available quizzes from the dashboard.

Step 3 — Start Quiz Attempt:
When a quiz is selected, the system sends a request to initialize an attempt. The backend creates an attempt record and stores the start time.

Step 4 — Attempt Questions:
The user answers the questions presented in the quiz interface. All questions are loaded from the backend, ensuring data consistency.

Step 5 — Anti-Cheat Monitoring:
During the attempt, the system monitors certain behaviors such as tab switching. If such behavior is detected, the attempt may be flagged as suspicious.

Step 6 — Submit Quiz:
Upon completion, the user submits the quiz. The selected answers are sent to the backend, where evaluation is performed server-side. The system calculates the score and updates the attempt record with results.

Step 7 — View Results:
After submission, the user can view their score and attempt details through the dashboard.

9.3 Attempt Lifecycle

The system follows a structured lifecycle for each quiz attempt:

Initialization:
An attempt record is created when the user starts a quiz, storing the start timestamp.

In Progress:
The user interacts with the quiz interface and selects answers.

Submission:
The user submits answers, which are processed on the server.

Evaluation:
The backend computes the score based on correct answers.

Finalization:
The attempt is marked as completed with score, total marks, and end timestamp. If suspicious behavior is detected, the attempt is flagged accordingly.

9.4 System Working (End-to-End Flow)

The overall working of the system can be summarized as follows:

The user interacts with the frontend interface built using React.

Requests are sent to backend API routes for processing.

The backend validates input and performs business logic.

Prisma ORM is used to interact with the PostgreSQL database.

Results are returned to the frontend and displayed to the user.

This flow ensures that all critical operations, especially evaluation and scoring, are handled securely on the server side, preventing manipulation from the client.

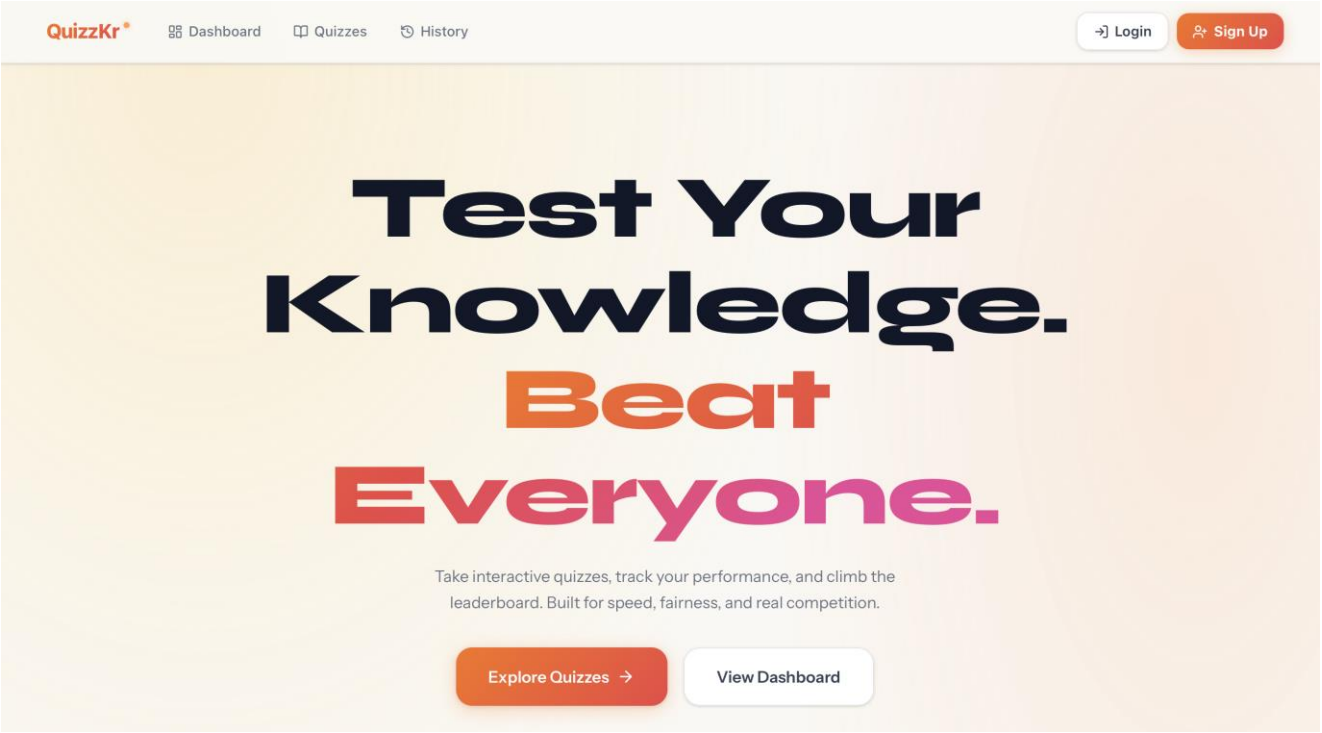
Chapter 10 — Result and Output

The following section describes the expected outputs at each stage of the Quizzkr platform. Output screenshots are to be placed in the labelled boxes below.

10.1 Landing Page

The landing page presents the Quizzkr branding, a brief description, and call-to-action buttons for Login and Register. The navigation bar shows the app name and links for Login, Register, and About.

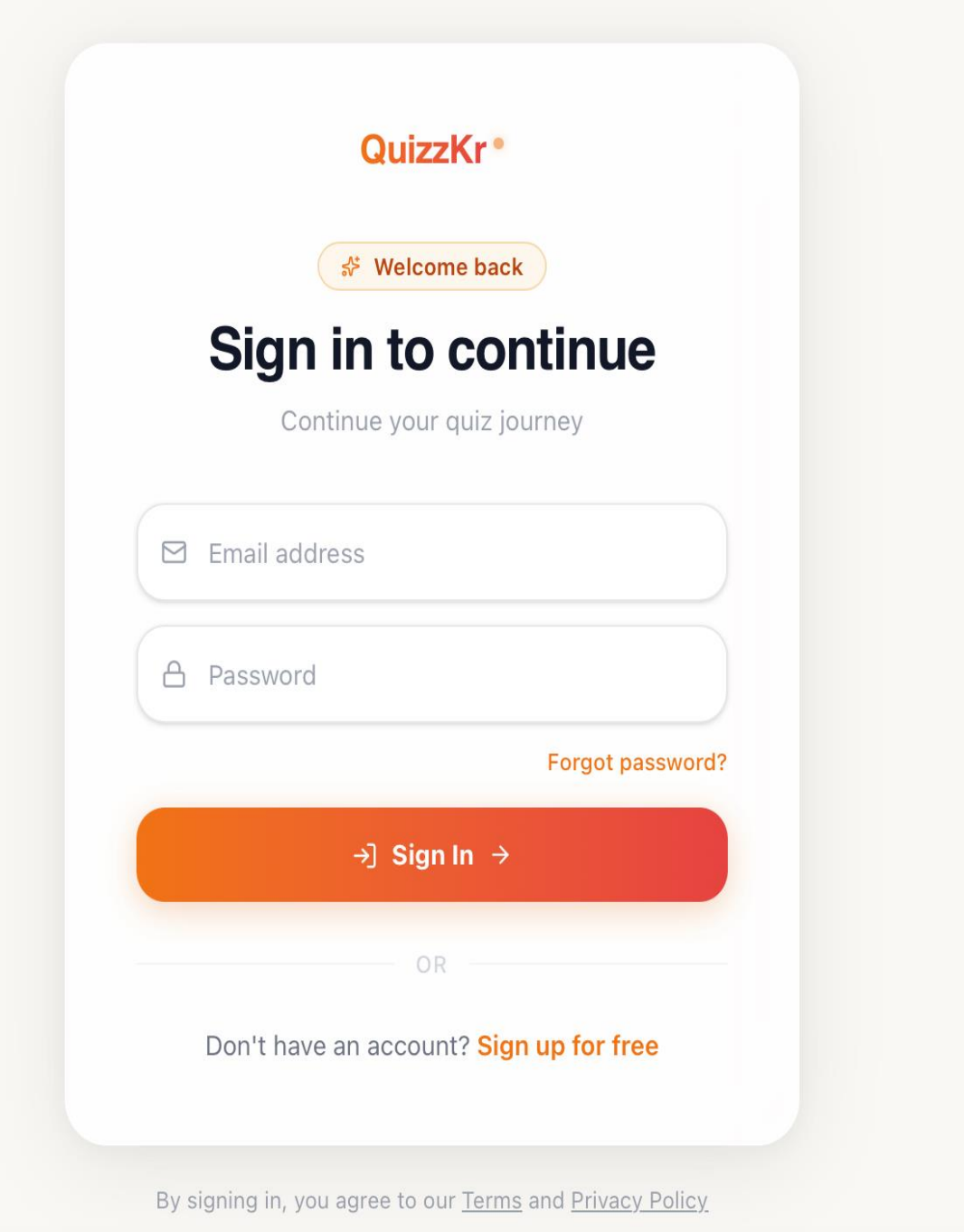
[Landing Page]



10.2 Registration

The registration form captures name, email, password, and role. On submission, the system creates the user, and redirects the user to Login page and after login the account is activated.

[Registration Form + Email]



QuizzKr

Free forever

Create your account

Join thousands of learners competing today

Full name

Email address

Password

Create Account →

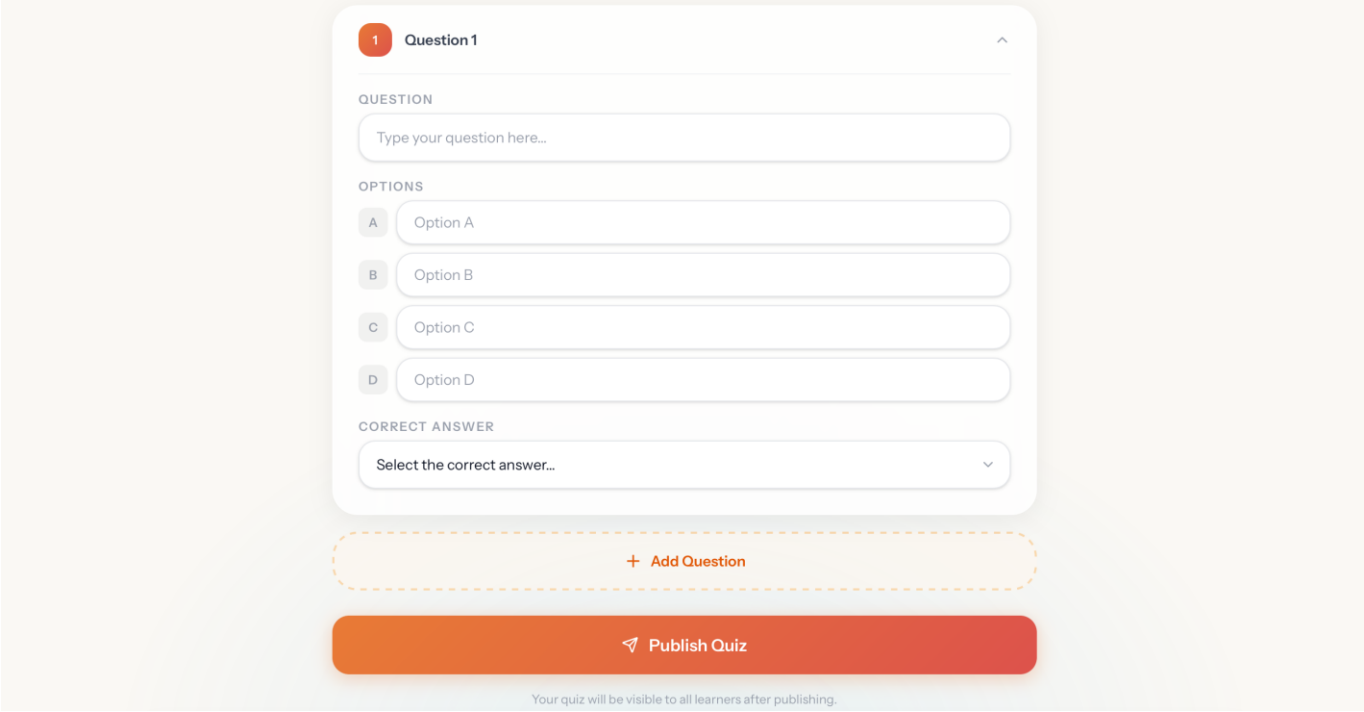
OR

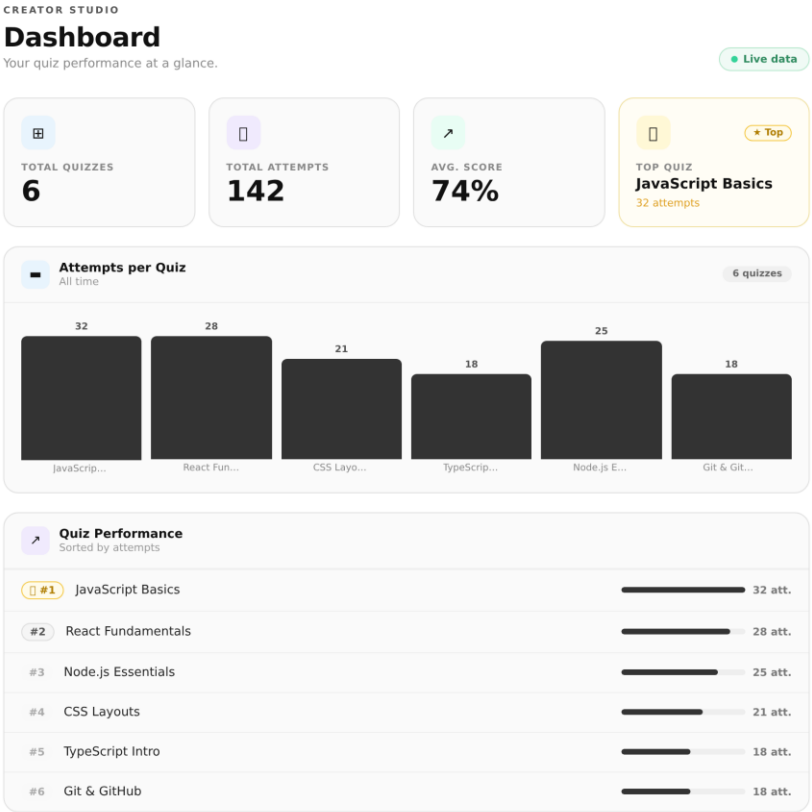
Already have an account? Sign in

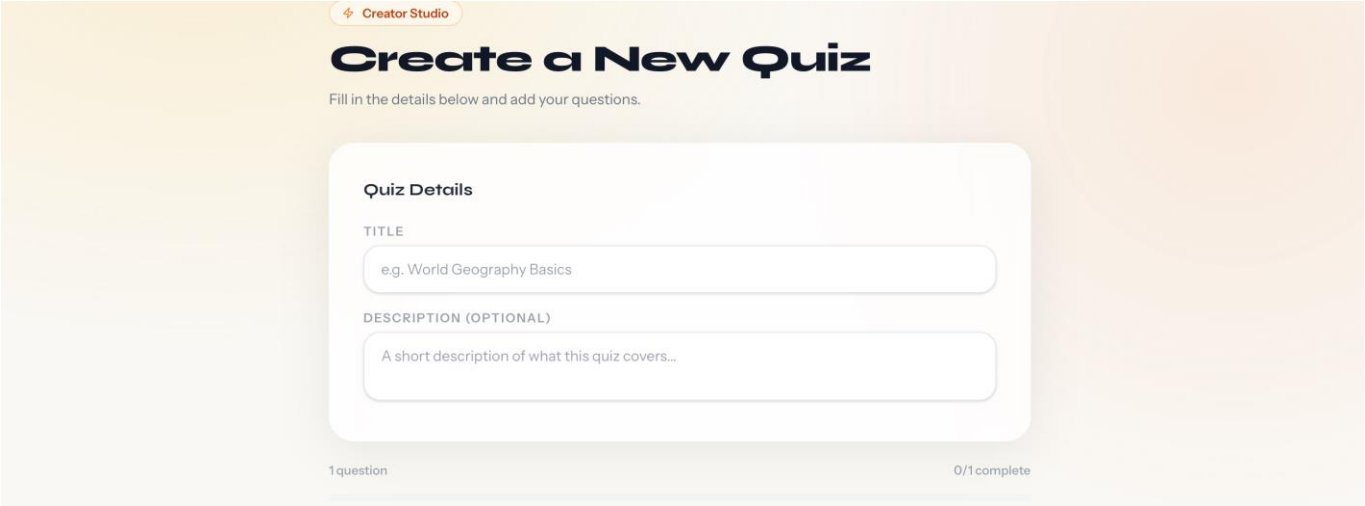
10.3 Teacher Dashboard — Create Quiz

After login, a Teacher sees their dashboard listing all created quizzes with question counts, attempt counts, and state badges (DRAFT/ACTIVE/COMPLETED). The Create Quiz wizard allows setting all quiz parameters and adding questions.

[Teacher Dashboard + Create Quiz]



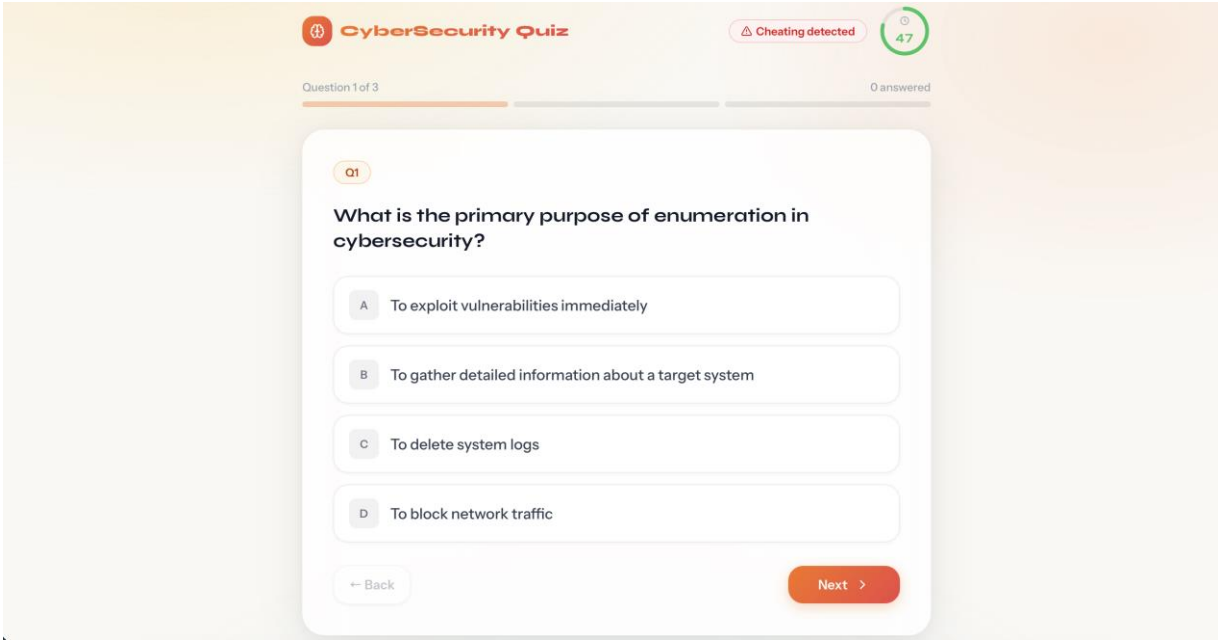


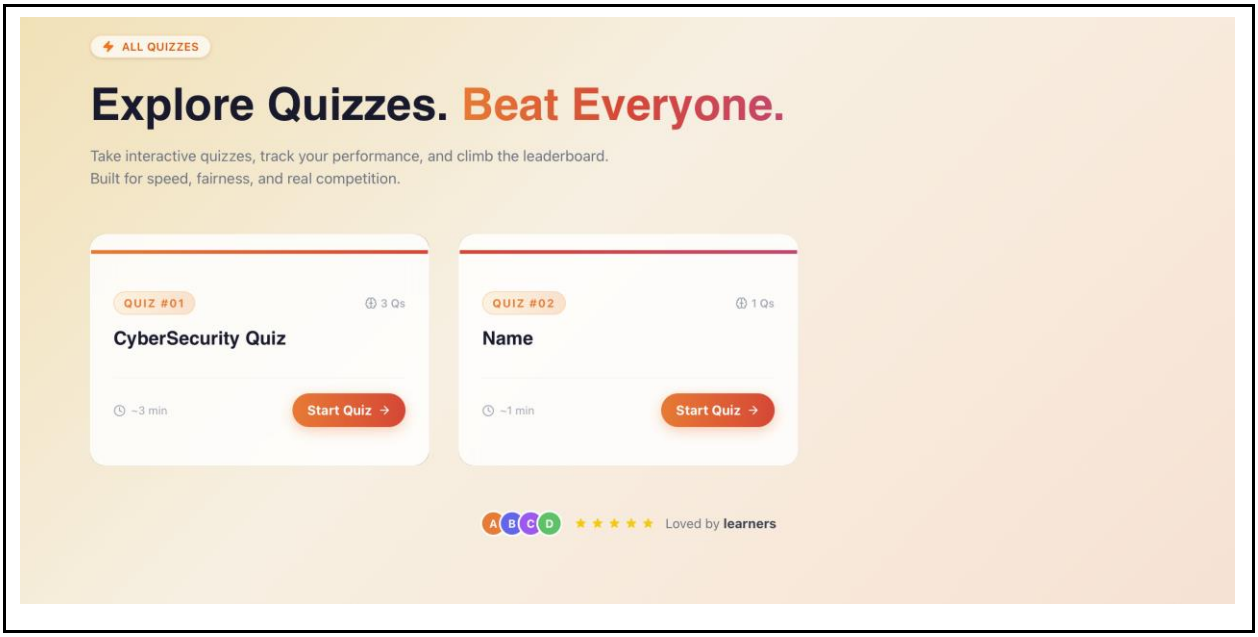


10.4 Student Quiz Attempt

The quiz attempt interface displays one question at a time with a countdown timer. The answer options are clearly laid out. A warning overlay appears when suspicious behaviour is detected.

[Quiz Attempt Interface with Timer]



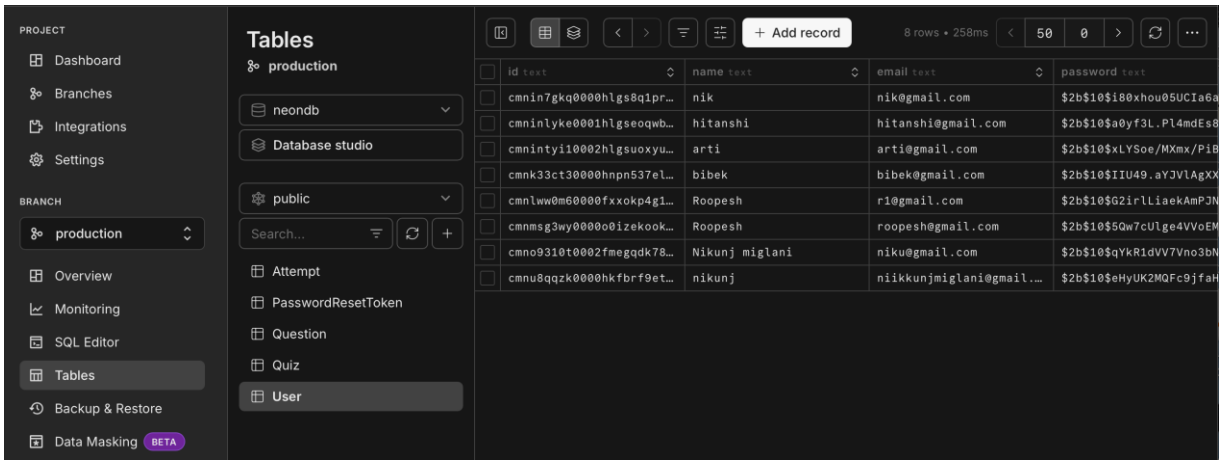


10.6 User Dashboard

[Analytics Dashboard]

The Neon DB console confirms successful table creation, migration execution, and live query results. All tables defined in the Prisma schema are visible in the Neon project dashboard.

[Neon DB Console showing Tables/Data]



Chapter 11 — Advantages and Limitations

11.1 Advantages

- Structured Relational Database Design:
  The system uses a well-defined relational schema with entities such as User, Quiz, Question, and Attempt. Relationships are enforced using foreign keys, and indexing is applied to improve query performance.

- Server-Side Evaluation for Security:
  All quiz evaluation and score computation are handled on the backend. The client only submits selected answers, preventing manipulation through browser tools.

- Role-Based Access Control:
  The platform supports multiple user roles (User, Creator, Admin), with access restrictions enforced at the API level. This ensures controlled interaction with system features.

- Modular API Architecture:
  The backend is structured using Next.js API route handlers, allowing separation of concerns between authentication, quiz management, and attempt handling.

- Integrated Anti-Cheating Detection:
  Basic anti-cheating mechanisms such as tab-switch detection and abnormal submission time analysis are implemented. Suspicious attempts are flagged for review.

- Secure Authentication and Recovery:
  Authentication is managed using NextAuth, and password reset functionality is implemented using token-based email verification via Nodemailer.

- Full-Stack Integration:
  The use of Next.js enables seamless integration of frontend and backend in a single codebase, simplifying development and deployment.

11.2 Limitations

- Basic Anti-Cheat Mechanisms:
  The current anti-cheating system is limited to simple behavioral checks such as tab switching and time-based detection. It does not include advanced proctoring features like webcam monitoring or IP tracking.

- Limited Question Format Support:
  The system currently supports only multiple-choice questions. Other formats such as descriptive or coding-based questions are not implemented.

- No Real-Time Monitoring:
  The system does not provide real-time tracking of user activity during a quiz. Suspicious behavior is only flagged after detection, not actively monitored.

- No Attempt Restrictions at Database Level:
  There is no strict database constraint enforcing limits such as one attempt per user per quiz, which could lead to multiple attempts if not handled carefully in the application logic.

- No Advanced Analytics:
  Analytics features are basic and limited to available dashboard metrics. Detailed insights such as question-level performance or statistical analysis are not implemented.

Chapter 12 — Future Enhancements

- Move score calculation to PostgreSQL stored procedures for stronger consistency
- Enforce one attempt per user per quiz at database level
- Add more question types (True/False, descriptive, coding)
- Improve anti-cheat with IP/device tracking and fullscreen enforcement
- Build a mobile app (React Native) using existing APIs
- Optimize performance using caching (e.g., Redis) and query tuning

Chapter 13 — Conclusion

The quiz platform developed in this project demonstrates the practical application of Database Management System concepts in building a structured and secure web-based assessment system. The implementation goes beyond a basic prototype by integrating a relational database, role-based access control, and a controlled evaluation pipeline.

The system applies key DBMS principles such as normalized schema design, foreign key relationships, indexing for performance, and structured data management using Prisma ORM with PostgreSQL. Each component of the system—from user management to quiz attempts—is designed to ensure data integrity and consistency.

A major strength of the platform is its server-side evaluation model, which prevents client-side manipulation of scores. Additionally, the inclusion of basic anti-cheating mechanisms such as tab-switch detection and abnormal submission time analysis highlights an effort to maintain fairness in assessments. The use of Next.js as a full-stack framework enables seamless integration between frontend and backend, while tools like NextAuth and Nodemailer provide secure authentication and account recovery mechanisms.

Although the system currently has limitations in terms of advanced anti-cheat features and extended question formats, it provides a solid foundation for further enhancements. Overall, the project successfully demonstrates how core DBMS concepts can be applied in designing a functional and reliable real-world application.

Chapter 14 — References

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill.

2. Prisma Documentation. (2024). Prisma ORM for TypeScript and Node.js. Retrieved from: https://www.prisma.io/docs

3. Neon DB Documentation. (2024). Serverless PostgreSQL. Retrieved from: https://neon.tech/docs

4. Next.js Documentation. (2024). App Router and API Route Handlers. Retrieved from: https://nextjs.org/docs

5. PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. Retrieved from: https://www.postgresql.org/docs/16/

6. Zod Documentation. (2024). TypeScript-first schema validation. Retrieved from: https://zod.dev

7. GeeksforGeeks. (2024). Database Management System. Retrieved from: https://www.geeksforgeeks.org/dbms/

8. Cloudflare. (2024). Turnstile CAPTCHA Documentation. Retrieved from: https://developers.cloudflare.com/turnstile/

9. Vercel. (2024). Vercel Platform Documentation. Retrieved from: https://vercel.com/docs

10. Google. (2024). Google OAuth 2.0 Documentation. Retrieved from: https://developers.google.com/identity/protocols/oauth2